SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1402 COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: *Construct top-down and bottomup parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 20 **Total Marks:100**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I am Midhuna A (192521302) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A. Midhuna

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Explanation	25%				
Application	20%				
Organization	15%				
Accuracy	10%				
Clarity	5%				

Faculty Incharge

Annexure A

SHORT ANSWER TEST

1. Construct language for the following Regular Expression:2

   (a|b)*a(a|b)|(a|b)

2. a) Illustrate the predictive parser for the following grammar. 4

   S → (L) | a
   L → L, S | S

3. Analyse, is it possible, by modifying the grammar in any way to construct predictive parser for the language of
   S → SS+ | SS* | a
   String: "*aa+a*" 4

4. Eliminate left recursion from the grammar $S \rightarrow Sa|Sb|c$ 2
5. The following grammar contains common prefixes, making it unsuitable for predictive parsing.

   A→ abcD | abcE | f

   Apply left factoring to remove the common prefixes and rewrite the grammar.5

6. Consider the following statement:10
   $$Degree - F = Degree - C * 1.8 + 32$$
   Show the representation of this statement after each phase of the compiler and explain.
7. Consider the following C code fragment:4

```
sum = 0;

for (i = 1; i < 5; i++)
    sum = sum + i;
```

   Show the representation of the above code after each phase of the compiler.2

8. Consider the following grammar.
   $$S \rightarrow aABb,\ A \rightarrow c|\varepsilon,\ B \rightarrow d|\varepsilon$$
   Calculate FIRST and FOLLOW, Obtain the Predictive Parsing Table.
   not. Show the parser moves for the input string **acdb.**
9. Write a simple Lex program to recognize and count identifiers in a given input.2
10. Write the rule to Eliminate Left Recursion and Left Factoring.2
11. What are the actions performed by a shift-reduce parser? Explain each action with a suitable example.3
12. Consider the following grammar:
    $$S \rightarrow pBB \rightarrow qB|r$$
    Find the leftmost derivation and construct the parse tree for the input string `pqqr`.3
13. Consider Grammar: E-> E+T|T T-> T*F|F F-> (E)|id
    Eliminate left recursion and find FIRST and FOLLOW 5
14. For the following grammar, determine whether the string `abcc` is accepted using shift-reduce parsing. Show all shift and reduce operations.

$$S \rightarrow ABA \rightarrow aA|aB \rightarrow cB|c$$

15. For the following grammar, verify whether the string **abbc** is valid using shift-reduce parsing.10

$$S \rightarrow aABA \rightarrow bA|bB \rightarrow c$$

Show the complete sequence of stack, input, and parser actions.

16. Consider the following grammar:10

$$S \rightarrow SS \rightarrow aABA \rightarrow bA|cB \rightarrow d$$

- Construct the canonical collection of LR(0) items.
- Construct the SLR(1) parsing table.

17. Consider the following grammar.

$$S \rightarrow iEtSS\,|a,\ S \rightarrow eS|\varepsilon,\ E \rightarrow b$$

Obtain the Predictive Parsing Table. 5

18. Do left factoring in the following grammar: 3

A-> aAB|aA|a

B-> bB|b

19. Construct the LL(1) Parsing Table for the following grammars.7

$$S \rightarrow ABCA \rightarrow a|\varepsilon B \rightarrow r|\varepsilon C \rightarrow b|\varepsilon$$

20. Construct the LL(1) Parsing Table for the following grammars.7

$$S \rightarrow 1S\#|qABCA \rightarrow a|bbDB \rightarrow a|\varepsilon C \rightarrow b|\varepsilon D \rightarrow c|\varepsilon$$

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers

Assessment Tool CO2 ①

Short answer test

Name : A. Midhuna

Register No : 192521302

Date : 4/8/26

Course No : CSA 1402

Course Name : compiler design

67/100

1. Regular Expression $(a|b)*a(a|b)|(a|b)$

2. a) predictive parser

$S \rightarrow (L) | a$

$L \rightarrow L,S | S$

2. Grammar way construct predictive parser language

$S \rightarrow SS+ | SS* | a$

String : "aa+a"

4. left recursion $S \rightarrow Sa / Sb | c$

2) $S \rightarrow (L) | a$

$L \rightarrow L,S | S$

$S \rightarrow (L)$

$S \rightarrow a$

$L \rightarrow S$

$L' \rightarrow ,SL' | \varepsilon$

3) Ans:

$S \rightarrow SS+ | SS* | a$

$S \rightarrow SS+$

$S \rightarrow SS*$

$S \rightarrow a$

First (S) → ~~Left recursion~~

~~S → aS'~~

First (S) = { (, a }

First (L) = { (, a }

First (L') = { , ε }

Follow(S) = { $,) }

Follow (L) = { $,) }

Follow (L') = { $,) }

Non Terminal	a	(	)	$	ε
S	S→a	S→(L)	S→(L)		
L	L→S	L→S			
L'			L'→,SL'	L'→ε	

7. C code fragments

```
sum = 0;
for(i=1; i<5; i++)
    sum = sum + i;
```

Predictive parsing

$A \rightarrow abcD \mid abcE \mid f$

6. Degree - F = Degree - C × 1.8 + 32

4) $S \rightarrow Sa \mid Sb \mid c$

$S \rightarrow cS'$

$S' \rightarrow aS' \mid \varepsilon$

$S' \rightarrow bS' \mid \varepsilon$

$S' \rightarrow aS' \mid bS' \mid \varepsilon$

5. Ans:

$A \rightarrow abcD \mid abcE \mid f$

$A \rightarrow abcD$

$A \rightarrow abcE$

$A \rightarrow f$

3. $S \rightarrow SS+ \mid SS* \mid a$

$S \rightarrow aS'$

$S' \rightarrow +S' \mid \varepsilon$

$S' \rightarrow *S' \mid \varepsilon$

$S' \rightarrow +S' \mid *S' \mid \varepsilon$

String = "aa+a"

$	aa+a$	$S \rightarrow aS'$
$S	aa+a$	$S' \rightarrow \varepsilon$
$aS'	aa+a$	$S \rightarrow a$
$a	a+a$	$S' \rightarrow \varepsilon$
$aS'a	a+a$	$S \rightarrow a$
$Sa	a+a$	$S \rightarrow a$
$SS	a+a$	$S \rightarrow aS'$
$aa+a'S	a	$S' \rightarrow +a'S$

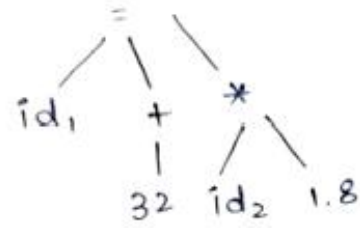
6) Degree -F = Degree -C * 1.8 + 32

<Degree -F> = <Degree -C> <*> <1.8> <+> <32>

Degree-F = id_1	identifier
Degree-C = id_2	identifier
1.8	constant
32	constant
*	multiplication factor

+ Addition factor

Syntax Analyser



Semantic analyser

$id_1 = id_2 * 1.8 + 32$

intermediate code generator

$id_1 = id_2$

$temp_1 = 1.8 + 32$

$temp_2 = Degree - C$

$temp_3 = temp_2 \times temp_1$

Code optimizer

$temp_1 = 1.8 + 32$

$temp_2 = id_2 * 1.8$

$temp_3 = temp_1 + temp_2$

Code function

MOV F, id_2, r_1

MOV F, id_2, 1.8+32

MOV F id_2, r_1

MOV F r_2, r_1

MOVF id, r,

8. Grammar $S \rightarrow aABb \quad A \rightarrow c \mid \varepsilon \quad B \rightarrow d \mid \varepsilon$

Calculate first & follow predictive parsing table
Input string acdb

9. Lex program count identifier input
10. Eliminate left recursion & Factoring
11. Shift reduced parser suitable eg
12. Ans:

8. $S \rightarrow aABb$
$A \rightarrow c \mid \varepsilon$
$B \rightarrow d \mid \varepsilon$

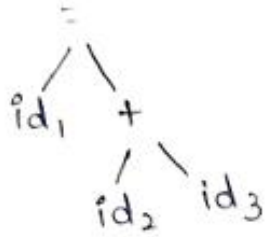
7. Ans:

lexical analyzer

Sum = id_1	identifier
Sum = id_2	identifier
=	equal operator
+	Addition operator
~~Sum = Sum + i~~	
i = id_3	identifier
;	Assignment operator

$id_1 = id_2 + i$ Identifier

Syntax analyzer



Semantic Analyzer

$id_1 = id_2 + id_3$

Intermediate code

$temp_1 = id_3$

$id_1 = temp_1 + id_2$

$id_2 = temp_2$

$temp_3 = temp_2 + temp_1$

code optimzer

$temp_1 = id_3$

$temp_2 = id_2$

$temp_3 = temp_2 + temp_3$

code function

MOV F id_3, r_2

MOV F id_2, r_1

Add id_2, r_2

MOV F r_1, r_2

MOV F id_1, r_1

8. ~~S → aABb A → c|ε.~~

12. S → pB
B → qB|r left most derivation parse tree input string pqqr.

13. E → E+T|T F → (E)|id left recursion first & follow

8) S → aABb
A → c|ε A → c B → d
B → d|ε A → ε B → ε

First (S) = {a} Follow (S) = {$, a}
First (A) = {c, ε} Follow (A) = {$, c, a}
First (B) = {d, ε} Follow (B) = {$, d, c, a}

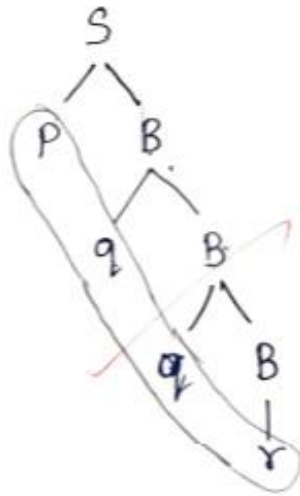
Non Terminal	a	c	d	ε	$
S	S→aABb				S→aABb
A	A→c|ε	A→c|ε			A→c|ε
B	B→d|ε	B→d|ε	B→d|ε	B→d|ε	B→d|ε

$S \rightarrow pB$

$B \rightarrow qB / r$

String pqqr

parse tree



14. string abcc shift reduce parsing & shift reduce operations

$S \rightarrow AB$

$A \rightarrow aA / a$

$B \rightarrow cB / c$

5. verify string abbc shift reducing parsing

$S \rightarrow aAB$

$A \rightarrow bA / b$

13. $E \to E+T|T$

$T \to T*F|F$

$F \to (E)|id$

$E \to TE'$

$E' \to +TE'|\varepsilon$

$T \to FT'$

$T' \to *FT'|\varepsilon$

$F \to (E)|id$

First(E) = {(, id}

First(E') = {+, ε}

First(T) = {(, id}

First(T') = {*, ε}

First(F) = {(, id}

Follow(E) = {$,)}

Follow(E') = {$,)}

Follow(T) = {+, $,)}

Follow(T') = {+, $,)}

Follow(F) = {*, +, $,)}

~~Non terminal~~	+	*	(	)	id	ε	$

$S \rightarrow AB$

$A \rightarrow aA \mid a$

$B \rightarrow CB \mid C$

String abcc

Stack	Input	output
$\$S$	abcc$\$$	Reduce $S \rightarrow AB$ shift
$\$AB$	abcc$\$$	$A \rightarrow aA$ (shift)
$\$aAB$	bcc$\$$	reduce $aA \rightarrow A$
$\$AB$	bcc$\$$	Shift
$\$ABb$	cc$\$$	shift
$\$ACB$	cc$\$$	shift
$\$ACBb$	cc$\$$	Reduce
$\$ABb$	c$\$$	Shift
$\$Acb$	c$\$$	Reduce
$\$Ab$	$\$$	shift
$\$ab$	$\$$	Not accepted

The abcc string not acapted.

15) $S \rightarrow aAB$

$A \rightarrow bA \mid b$

$B \rightarrow C$

$S \rightarrow aAB$
$A \rightarrow bA$
$A \rightarrow b$
$B \rightarrow c$

String abbc

Stack	Input	output
$	abbc$	shift
$a	bbc$	shift
$ab	bc$	~~shift~~ Reduce $b \rightarrow A$
$aA	bc$	Reduce ($A \rightarrow bA$)
$abA	bc$	Shift
$abAb	c$	Reduce $bA \rightarrow A$
$aAb	c$	Reduce $bA \rightarrow A$
$aA	c$	shift
$aAc	$	Reduce $C \rightarrow B$
$aAB	$	Reduce $aAB \rightarrow S$
$S	$	accepted

left factoring

$A \rightarrow aAB|aA|a$

$B \rightarrow bB|b$

19. LL(1) parsing table

$S \rightarrow ABC$

$A \rightarrow a|\varepsilon$

$B \rightarrow x|\varepsilon$

$C \rightarrow b|\varepsilon$

20) LL(1) parsing Table $S \rightarrow ISt\#|q ABC$

$A \rightarrow a|bbD$

$B \rightarrow a|\varepsilon$

$C \rightarrow b|\varepsilon$

$D \rightarrow C|\varepsilon$

17. $S \rightarrow$ 'iEtSS'|a, $S' \rightarrow$ es| ε. $E \rightarrow b$

predictive parsing table

10. left recursion parsing:

* Shift reduce parsing are used in the stack emplementation

* The left recursion parsing check the Terminal and Non terminal of the parsing.

$S \rightarrow AB$

$A \rightarrow aA \mid a$

$B \rightarrow CB \mid C$

String abcc

Stack	Input	output
$\$S$	abcc$	Reduce S→AB shift
$\$AB$	abcc$	A→aA (shift)
$\$aAB$	bcc$	reduce aA→A
$\$AB$	bcc$	Shift
$\$ABb$	cc$	shift
$\$ACB$	cc$	shift
$\$ACBb$	cc$	Reduce
$\$ABb$	c$	Shift
$\$Acb$	c$	Reduce
$\$Ab$	$	shift
$\$ab$	$	Not accepted

The abcc string not acapted.

15) $S \rightarrow aAB$

$A \rightarrow bA \mid b$

$B \rightarrow c$

16.

$S' \to S$
$S \to aAB$
$A \to bA|c$
$B \to d$

Goto $(I_0, S') = I_1$
$S' \to .S$
$S \to .aAB$

Goto $(I_0, A) = I_2$
$A \to .bA|c$
~~$A \to$~~

Goto $(I_0, B) = I_3$
$B \to .d$
$B \to d.$

Goto $(I_0, A) = I_4$
$A \to c.$

Goto $(I_1, a) = I_5$
$S \to aAB$
$S \to acd$
$S \to .acd$

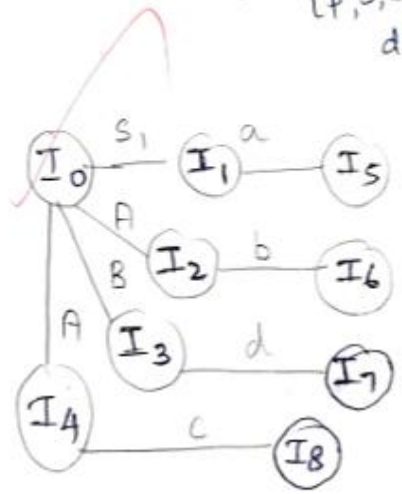
Goto $(I_2, b) = I_6$
$A \to bA$
$A \to .bA$

Goto $(I_3, d) = I_7$
$B \to d \Rightarrow B \to .d$

Goto $(I_4, c) =$
$A \to .c$

First $(S') = \{a\}$
First $(S) = \{a\}$
First $(A) = \{b, c\}$
First $(B) = \{d\}$

Follow $(S') = \{\$\}$
Follow $(S) = \{\$, a\}$
Follow $(A) = \{\$, b, c, d\}$
Follow $(B) = \{\$, b, c, a, d\}$



	Active				Shift			
	a	b	c	d	$	S'	A	B
0						1	2	6
1	S_5							
2		S_6	r_3	r_3				
3	r_3	r_3		S_3				
4								
5	S_5							
6		S_6						
7				S_8				
8								

9